DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

ARIMA | Time Series | Differencing | LAG() | Forecasting | Statistics

Project Summary

Implements the data preparation steps for an ARIMA(p,d,q) model entirely in SQL. Applies first and second-order differencing to remove trend and non-stationarity from the claims metric series, then generates autoregressive lag features (lags 1, 2, and 3) as predictors. The output is ready for downstream model fitting or feature engineering pipelines.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_arima
Layer:	Intermediate
Materialization:	view
ARIMA Components:	d=1 (first difference), d=2 (second difference), AR lags 1-3
Key Outputs:	diff_1, diff_2, ar_lag_1, ar_lag_2, ar_lag_3 per entity/date
Audience:	Data Scientists, Actuaries, Forecasting teams

2 BUSINESS PROBLEM

Healthcare claims data is non-stationary -- it grows over time as populations age and medical costs rise. Standard regression and averaging techniques assume a stable mean, which is violated by trending series. ARIMA resolves this by differencing the series to achieve stationarity, then modeling the autocorrelated structure of the residuals. This model prepares all the features needed for an ARIMA(3,1,0) specification without leaving Snowflake.

- First differencing (d=1): X_t - X_{t-1} removes linear trend from the series.
- Second differencing (d=2): Removes quadratic trend for accelerating/decelerating series.
- AR lags (p=3): Three lags of the differenced series capture autocorrelation structure.
- Pure SQL: All transformations run in Snowflake -- no Python or external tools needed.
- Partitioned by entity_id: Each entity's differencing is computed independently.

3 TECHNICAL EXPLANATION

- source_data CTE: Pulls entity_id, period_date, and metric_value from the claims source.
- differenced_1 CTE: Computes diff_1 = metric_value - LAG(metric_value,1) and diff_2 = diff_1 - LAG(diff_1,1) using LAG() window functions partitioned by entity_id.
- ar_features CTE: Generates AR lag features: ar_lag_1 = LAG(diff_1,1), ar_lag_2 = LAG(diff_1,2), ar_lag_3 = LAG(diff_1,3). These are the p=3 autoregressive predictors.
- First rows per entity will have NULL lags (warm-up) -- expected ARIMA behavior, filter in consuming model.
- Extend ar_lag_N for higher p values; extend diff_N for d > 2.

Key Techniques: LAG() for differencing, LAG() for AR lags, PARTITION BY entity, ARIMA(p,d,q) SQL pattern

4 FULL MODEL CODE

```
{{ config(materialized='view') }}

with source_data as (
    select
        entity_id,
        period_date,
        metric_value
    from {{ source('healthcare', 'claims') }}
),

-- d=1 differencing: (1-B)^1 X_t = X_t - X_{t-1}
differenced_1 as (
    select
        entity_id,
        period_date,
        metric_value,
        metric_value - lag(metric_value, 1) over (
            partition by entity_id order by period_date
        ) as diff_1,
        metric_value
            - lag(metric_value, 1) over (partition by entity_id order by period_date)
            - (
                lag(metric_value, 1) over (partition by entity_id order by period_date)
                - lag(metric_value, 2) over (partition by entity_id order by period_date)
            ) as diff_2   -- d=2
    from source_data
),

-- AR lag features: p lags of the once-differenced series
ar_features as (
    select
        entity_id,
        period_date,
        metric_value,
        diff_1,
        lag(diff_1, 1) over (partition by entity_id order by period_date) as ar_lag_1,
        lag(diff_1, 2) over (partition by entity_id order by period_date) as ar_lag_2,
        lag(diff_1, 3) over (partition by entity_id order by period_date) as ar_lag_3
    from differenced_1
)

select *
from ar_features
order by entity_id, period_date
```

5 EXAMPLE OUTPUT

entity_id	period_date	metric_value	diff_1	diff_2	ar_lag_1	ar_lag_2	ar_lag_3
E-001	2025-01	4250.75	NULL	NULL	NULL	NULL	NULL
E-001	2025-02	4380.20	129.45	NULL	NULL	NULL	NULL
E-001	2025-03	4520.80	140.60	11.15	129.45	NULL	NULL

E-001	2025-04	4390.10	-130.70	-271.30	140.60	129.45	NULL
E-001	2025-05	4610.55	220.45	351.15	-130.70	140.60	129.45

** NULL values in early rows are expected warm-up periods -- filter with WHERE ar_lag_3 IS NOT NULL in consuming models.*